Metody Programowania Równoległego
MPI

Jakub Ciszewski

Marzec 2026

1 Część 1

1.1 Wstęp

Celem pierwszej części ćwiczenia było zbadanie wydajności komunikacji punkt-punkt w standardzie MPI (*Message Passing Interface*). Kluczowym aspektem pomiarów było wyznaczenie dwóch podstawowych parametrów sieciowych: opóźnienia (*latency*) oraz przepustowości (*bandwidth*) w zależności od rozmiaru przesyłanego komunikatu.

W ramach zadań zaimplementowano mechanizm typu *Ping-Pong* pomiędzy dwoma procesami. Pomiary zostały przeprowadzone dla różnych wariantów komunikacji:

- **Komunikacja synchroniczna (`Ssend`):** gdzie proces nadawcy jest blokowany do momentu rozpoczęcia odbierania danych przez odbiorcę.
- **Komunikacja buforowana (`Bsend`):** wykorzystująca zadeklarowany bufor pamięci, co pozwala nadawcy na kontynuowanie pracy natychmiast po skopiowaniu danych do bufora.

Badania wykonano w dwóch konfiguracjach środowiskowych: lokalnie (w ramach jednego węzła obliczeniowego) `vnode-01` oraz sieciowo (pomiędzy dwoma fizycznie oddzielnymi jednostkami klastra: `vnode-05` i `vnode-06`).

1.2 Dane pomiarowe

Dane zbierano dla wielu rozmiarów wiadomości, zaczynając od 1 B, a kończąc na 8 MB (stosując skok o potędze liczby 2), co pozwoliło na zobrazowanie charakterystyki pracy sieci.

1.2.1 Parametry pomiaru

Dla każdego rozmiaru wiadomości wykonano serię $N = 1000$ powtórzeń testu typu *Ping-Pong*. Czas mierzony był przy użyciu funkcji `MPI.Wtime()`, a wynik

końcowy dla pojedynczej próbki obliczano jako połowę całkowitego czasu rundy:

$$t_{avg} = \frac{t_{end} - t_{start}}{2}$$

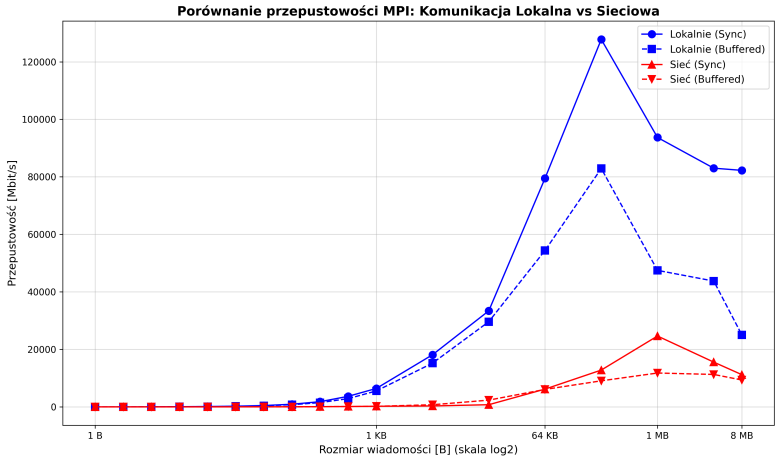
1.2.2 Struktura wyników

Wyniki zostały zapisane w formacie plików CSV. Każdy plik zawiera następujące kolumny:

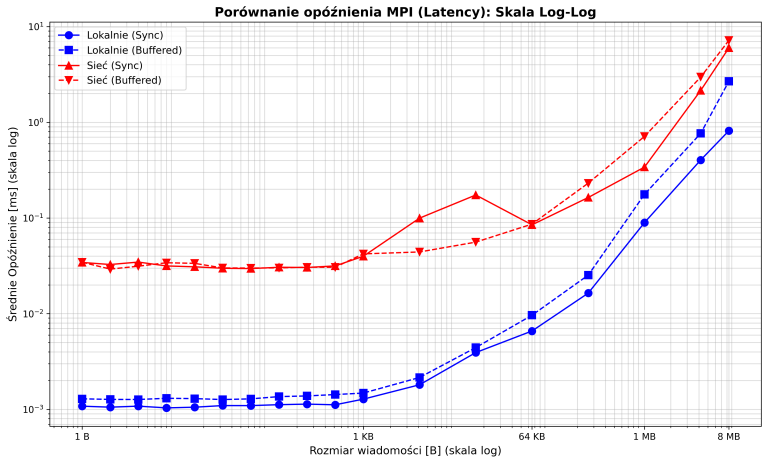
- **size_bytes**: rozmiar przesyłanego bufora w bajtach,
- **latency_ms**: średnie opóźnienie przesyłu wyrażone w milisekundach,
- **bandwidth_mbit_s**: obliczona przepustowość wyrażona w megabitach na sekundę (Mbit/s).

1.3 Wyniki i wnioski

Na podstawie przeprowadzonych pomiarów wygenerowano wykresy porównawcze dla komunikacji lokalnej oraz sieciowej, z uwzględnieniem dwóch trybów przesyłania danych.



Wykres 1: Porównanie przepustowości w funkcji rozmiaru wiadomości dla komunikacji lokalnej oraz międzywęzłowej.



Wykres 2: Charakterystyka opóźnienia w skali logarytmicznej dla różnych trybów komunikacji.

1.3.1 Analiza przepustowości

Wykres 1 przedstawia zależność szybkości transferu od rozmiaru komunikatu.

- **Komunikacja lokalna:** Osiąga znacznie wyższe wartości. Wynika to z faktu, że dane są przesyłane w obrębie pamięci współdzielonej i cache'u procesora, co eliminuje narzut sieciowy.
- **Komunikacja sieciowa:** Wykazuje wyraźne nasycenie przy większych komunikatach, osiągając asymptotę wyznaczoną przez fizyczną przepustowość interfejsu sieciowego klastra.
- **Wpływ rozmiaru:** Dla małych komunikatów przepustowość w obu przypadkach jest niska. Wynika to z faktu, że całkowity czas operacji jest zdominowany przez stałe opóźnienie związane z narzutem programowym i synchronizacją protokołu MPI. Czas potrzebny na fizyczny transfer tak niewielkiej ilości danych jest pomijalny w stosunku do czasu przygotowania komunikacji, przez co efektywne wykorzystanie dostępnego pasma jest niemożliwe.

1.3.2 Analiza opóźnień

Wykres 2 pozwala na dokładną analizę czasu dostępu.

- **Opóźnienie krótkiego komunikatu:** Dla wiadomości 1-bajtowej zmierzone opóźnienie w komunikacji sieciowej wynosi ok. 0.034 ms, podczas gdy lokalnie jest ono o rząd wielkości mniejsze - ok. 0.001 ms. Różnica ta wynika z konieczności przejścia sygnału przez sieć.

- **Stały narzut:** W zakresie małych wiadomości opóźnienie utrzymuje się na niemal stałym poziomie. Świadczy to o dominacji czasu narzutu programowego i sprzętowego nad czasem samej transmisji bitów.

1.3.3 Porównanie trybów Ssend i Bsend

Z pomiarów wynika, że:

- **Tryb synchroniczny (`Ssend`):** Zapewnia bezpieczniejszą komunikację, ale w specyficznych warunkach może być nieco wolniejszy ze względu na konieczność pełnej synchronizacji procesów.
- **Tryb buforowany (`Bsend`):** Pozwala na "ukrycie" części opóźnień poprzez natychmiastowe kopiowanie danych do zadeklarowanego bufora. Jest to szczególnie widoczne przy mniejszych komunikatach, o ile dostępna jest wystarczająca ilość pamięci bufora.

1.3.4 Wnioski

1. Wydajność systemów rozproszonych MPI jest ściśle uzależniona od topologii połączeń.
2. Dla efektywnego wykorzystania sieci w algorytmach równoległych należy dążyć do przesyłania mniejszej liczby większych pakietów danych zamiast wielu małych wiadomości, co minimalizuje wpływ opóźnienia stałego.
3. Wybór trybu komunikacji (`Bsend`/`Ssend`) powinien być podyktowany dostępnością pamięci RAM oraz wymaganiami dotyczącymi synchronizacji algorytmu.

2 Część 2

2.1 Wstęp

Drugą część ćwiczenia poświęcono analizie wydajnościowej algorytmu równoległego na przykładzie obliczeń numerycznych metodą Monte Carlo. Głównym celem było zbadanie skalowalności rozwiązania oraz efektywności operacji zbiorowych w standardzie MPI.

Problem badawczy polegał na wyznaczeniu przybliżonej wartości liczby π poprzez losowanie punktów w kwadracie o boku $2r$ i zliczanie tych, które trafiły w obręb wpisanego w niego koła. W odróżnieniu od komunikacji punkt-punkt badanej w pierwszej części, w tym zadaniu kluczową rolę odegrały funkcje:

- `MPI_Reduce`**:** wykorzystana do sumowania wyników cząstkowych ze wszystkich procesów do procesu głównego.
- `MPI_Gather`**:** zastosowana do zebrania indywidualnych czasów wykonania każdego z procesów.

Badania przeprowadzono na klastrze **Ares**, testując zachowanie algorytmu dla różnej liczby rdzeni procesora. Analiza została podzielona na dwa główne warianty skalowania:

- **Skalowanie silne:** badanie czasu wykonania przy stałym całkowitym rozmiarze problemu i zwiększającej się liczbie zasobów obliczeniowych.
- **Skalowanie słabe:** badanie wydajności przy zachowaniu stałego obciążenia na pojedynczy rdzeń, co wiąże się ze wzrostem rozmiaru problemu proporcjonalnie do liczby procesów.

2.2 Metodyka i parametry pomiarowe

Kluczowym elementem badania było dobranie rozmiarów problemu (liczby punktów N) w taki sposób, aby zaobserwować różnice w charakterystykach wydajnościowych. Zgodnie z wytycznymi, wyznaczono trzy scenariusze badawcze:

1. **Rozmiar mały(small):** dobrany tak, aby czas wykonania na jednym rdzeniu był wyraźnie niższy od 1 sekundy, lecz mierzalny ($N_{small} = 5 \cdot 10^6$).
2. **Rozmiar duży(big):** dobrany tak, aby przy maksymalnej liczbie rdzeni ($n = 12$) czas obliczeń wynosił co najmniej 60 sekund ($N_{large} = 6 \cdot 10^{10}$).
3. **Rozmiar średni(medium):** wyznaczony jako przybliżona średnia geometryczna z powyższych wartości ($N_{medium} \approx 5,5 \cdot 10^8$).

W przypadku **skalowania silnego**, powyższe wartości N stanowiły stałą pulę punktów dzieloną pomiędzy n procesów ($N_{local} = N/n$).

W przypadku **skalowania słabego**, aby zachować stałe obciążenie jednostkowe, przyjęto wartości bazowe przypadające na jeden rdzeń (N_{per_core}), które następnie skalowano liniowo wraz z liczbą procesów ($N_{total} = N_{per_core} \cdot n$). Wartości te przedstawiono w poniższej tabeli:

Rozmiar	**Skalowanie silne** (N_{total})	**Skalowanie słabe** (N_{per_core})
Mały	$5 \cdot 10^6$	$4,16 \cdot 10^5$
Średni	$5,5 \cdot 10^8$	$4,5 \cdot 10^7$
Duży	$6 \cdot 10^{10}$	$5 \cdot 10^9$

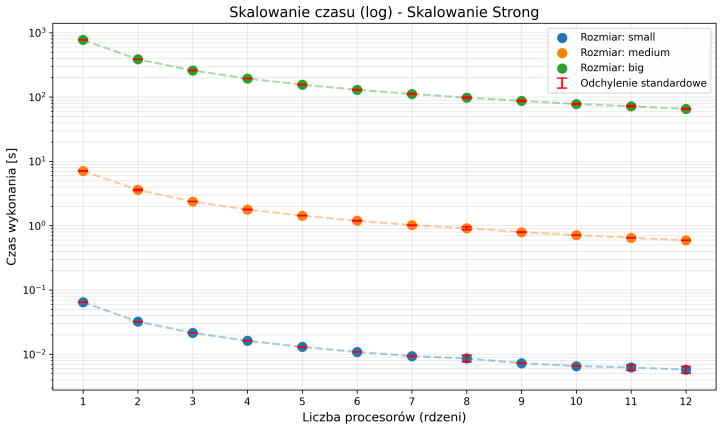
Table 1: Zestawienie liczby punktów Monte Carlo dla poszczególnych wariantów eksperymentu.

Ze względu na bardzo dużą liczbę punktów w rozmiarze dużym, w implementacji wykorzystano typ `long long`. Pomiary czasu przeprowadzono z wykorzystaniem funkcji `MPI_Wtime()`, biorąc pod uwagę czas najdłużej pracującego procesu (operacja `MAX` na zmierzonych czasach lokalnych).

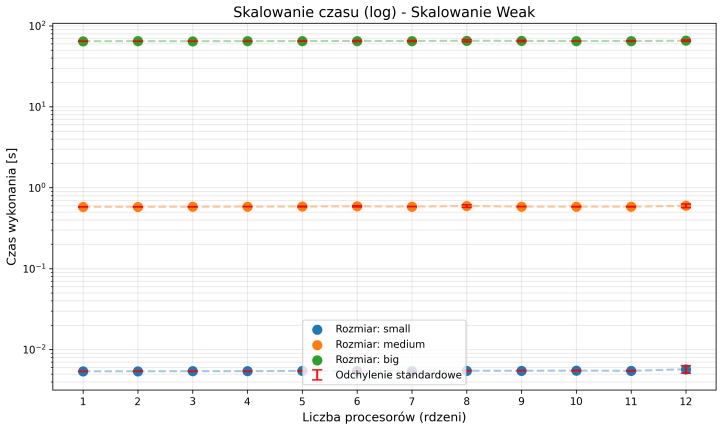
2.3 Analiza porównawcza metryk wydajności

W tej sekcji zestawiono wyniki skalowania silnego oraz słabego.

2.3.1 Czas wykonania



Wykres 3: Czas wykonania dla skalowania silnego (oś Y w skali logarytmicznej).

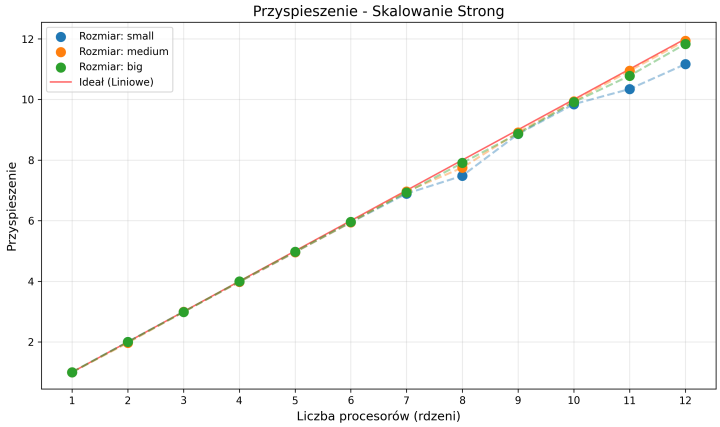


Wykres 4: Czas wykonania dla skalowania słabego (oś Y w skali logarytmicznej).

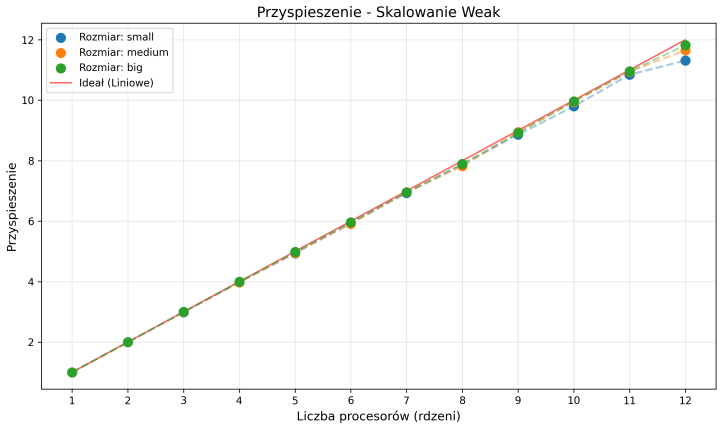
Analiza porównawcza: W skalowaniu silnym (Wyk. 3) obserwujemy niemal idealny spadek czasu wraz z dodawaniem procesorów dla wszystkich serii. Stałe nachylenie prostej w skali logarytmicznej potwierdza wysoką skalowalność algorytmu.

W skalowaniu słabym (Wyk. 4) idealnym wynikiem byłaby linia pozioma. Dla wszystkich rozmiarów czas jest niemal stały, co dowodzi, że narzut komunikacyjny MPI rośnie bardzo powoli i nie dominuje nad obliczeniami nawet przy 12 rdzeniach. Niewielkie odchylenia standardowe są minimalne, co świadczy o stabilności klastra Ares.

2.3.2 Przyspieszenie



Wykres 5: Przyspieszenie dla skalowania silnego.



Wykres 6: Przyspieszenie dla skalowania słabego.

Analiza porównawcza: W skalowaniu silnym oraz słabym obserwujemy niemal idealny wzrost liniowy (Wyk. 5 i 6).

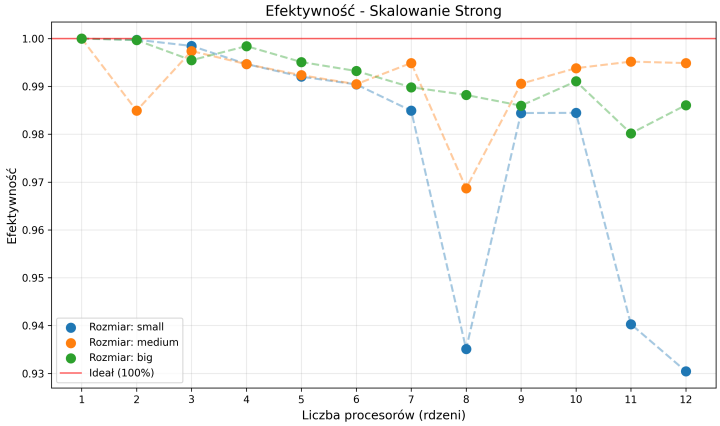
W badaniu wyznaczono **przyspieszenie względne**

$$S_{STRONG} = \frac{T_{MPI}(1)}{T_{MPI}(n)}, S_{WEAK} = \frac{n * T_{MPI}(1)}{T_{MPI}(n)}$$

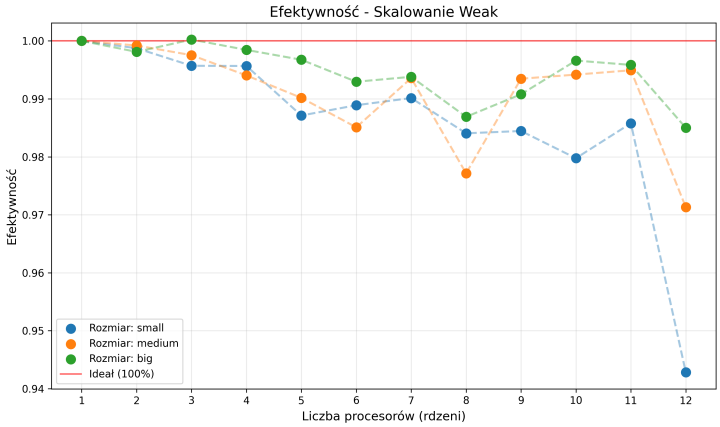
, co pozwala ocenić skalowalność samej logiki algorytmu wewnątrz środowiska MPI, pomijając stały narzut inicjalizacji biblioteki. W ogólnym przypadku

przyspieszenie bezwzględne (wyznaczane względem zoptymalizowanego kodu sekwencyjnego bez biblioteki MPI) mogłoby być nieco niższe ze względu na narzuty komunikacyjne i kopiowanie pamięci, jednak przy tak dużej intensywności obliczeniowej jak w rozmiarze *Big*, różnica ta staje się pomijalna.

2.3.3 Efektywność (Efficiency)



Wykres 7: Efektywność dla skalowania silnego.

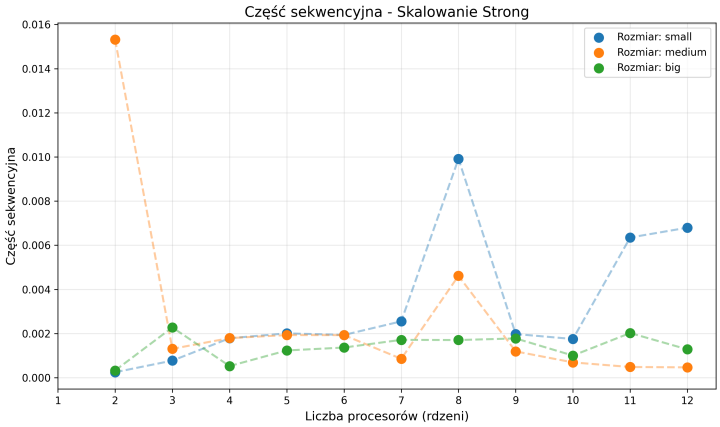


Wykres 8: Efektywność dla skalowania słabego.

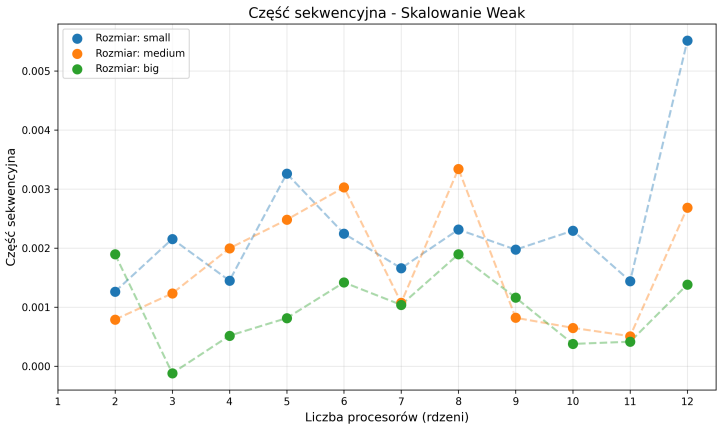
Analiza porównawcza: Efektywność dla dużych danych utrzymuje się powyżej 98%(Wyk. 7 i 8). Spadek efektywności w wariancie *strong/small* przy 12 rdzeniach jest wynikiem zbyt małej ilości pracy przypadającej na proces w stosunku do czasu potrzebnego na komunikację `MPI_Reduce`. W wariancie skalowania

słabego (*weak/small*) zaobserwowano niewielki spadek efektywności wraz ze wzrostem liczby rdzeni (z 100% do ok. 94%). Wynika to z faktu, że przy bardzo krótkim czasie obliczeń jednostkowych, stały narzut komunikacyjny operacji zbiorowej `MPI_Reduce` (której koszt rośnie logarytmicznie wraz z liczbą procesów) zaczyna stanowić mierzalny odsetek całkowitego czasu wykonania.

2.3.4 Część sekwencyjna



Wykres 9: Część sekwencyjna dla skalowania silnego.



Wykres 10: Część sekwencyjna dla skalowania słabego.

Analiza porównawcza: Niskie i stabilne wartości części sekwencyjnej (Wyk. 9 i 10) potwierdzają, że narzut systemowy i komunikacyjny jest stały i nie rośnie

gwałtownie wraz z liczbą procesorów, co pozwala na efektywne wykorzystanie zasobów klastra Ares.

2.4 Wnioski końcowe

1. Algorytm obliczania liczby π metodą Monte Carlo cechuje się **naturalną równoległością**, co potwierdzają uzyskane wyniki efektywności bliskie 100%.

2. Rozmiar problemu ma kluczowe znaczenie dla obserwowalnej wydajności. Przy małych danych narzut komunikacyjny stanowi istotny procent całkowitego czasu. Przy dużych danych czas obliczeń dominuje, pozwalając na niemal liniowe skalowanie.

3. Operacja zbiorowa `MPI_Reduce` jest bardzo wydajnym sposobem agregacji danych – mimo wzrostu liczby procesów do 12, czas potrzebny na zsumowanie wyników nie wpłynął znacząco na pogorszenie metryk.